

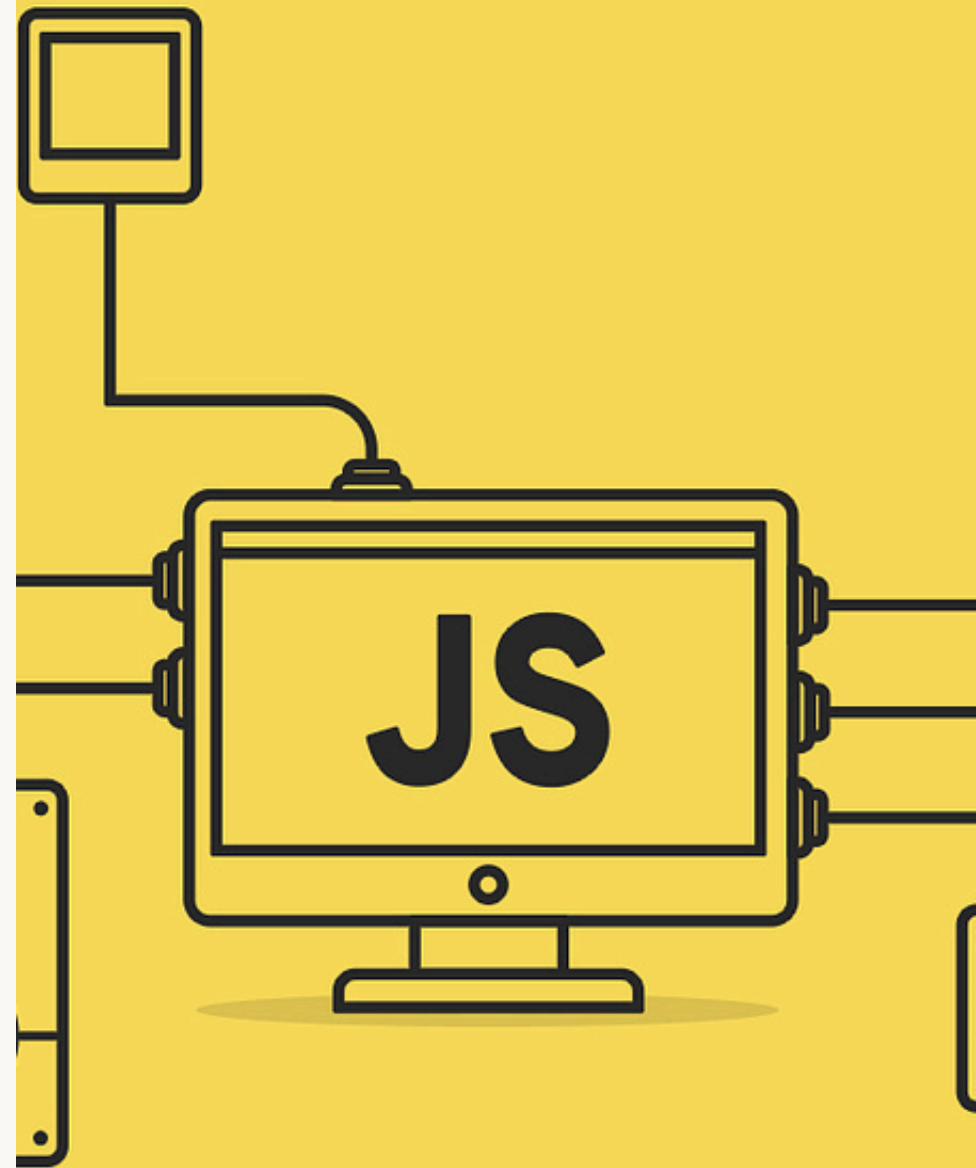
Funções em JavaScript

Uma introdução prática e acessível para quem está começando a programar — do básico às closures.

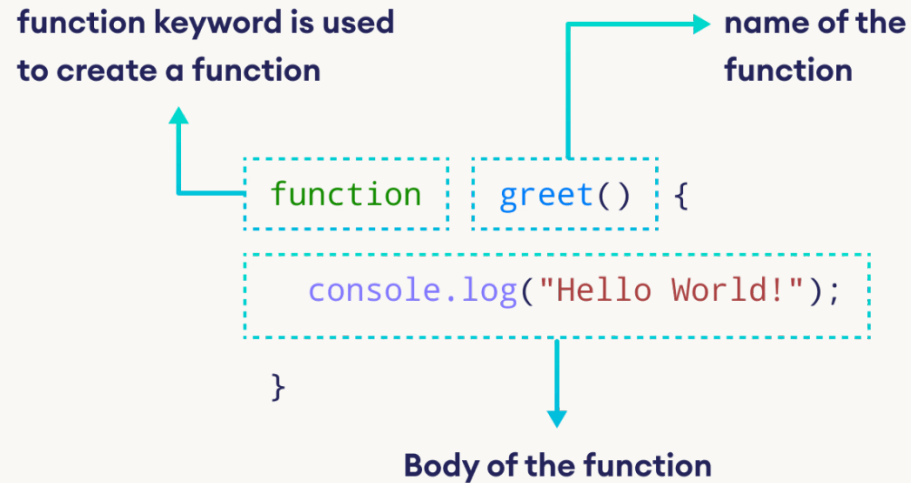
JAVASCRIPT

INICIANTE

FUNDAMENTOS



O que é uma Função?



Uma função é um **bloco de código** definido uma única vez, mas que pode ser **executado quantas vezes for necessário**.

Parametrizada

Recebe valores de entrada chamados **argumentos**, processados por **parâmetros**.

Retorna um valor

Usa `return` para devolver um resultado ao código que a chamou.

É um objeto

Em JavaScript, funções são objetos de primeira classe — podem ser atribuídas a variáveis e passadas como argumento.



Funções são objetos

No JavaScript, funções são **objetos de primeira classe**: podem ser atribuídas a **variáveis**, passadas como **argumentos** para outras funções e até ter propriedades definidas nelas. Isso abre um mundo de possibilidades.

Aninhamento

Funções podem ser definidas dentro de outras funções, tendo acesso a todas as variáveis do escopo externo.

Closures

Esse aninhamento com acesso ao escopo externo é o que torna possível as poderosas **closures** em JavaScript.

Declaração de Funções

A forma mais tradicional de criar uma função usa a palavra-chave `function`, seguida pelo nome, pela lista de parâmetros entre parênteses e pelo corpo entre chaves. A instrução `return` especifica o valor retornado.

```
function hello(nome) {  
  return console.log(`Olá, ${nome}! Seja bem-vindo!`);  
}
```

Depois de declarada, a função pode ser **invocada** pelo seu nome (identificador) em qualquer ponto do programa:

```
hello("Ada"); // → Olá, Ada! Seja bem-vindo!
```

- ❗ A declaração de função é **icada** (*hoisted*) para o topo do escopo, o que significa que você pode chamá-la antes mesmo de definí-la no código.

Expressões de Funções

Funções também podem ser criadas como **expressões** e atribuídas a variáveis ou constantes. Ao contrário das declarações, expressões de função **não precisam ter nome** — são chamadas de funções anônimas.

```
let cumprimentar = function (nome) {  
  return console.log(`Olá, ${nome}! Seja bem-vindo!`);  
}
```

Nesse caso, usamos o **nome da variável** para invocar a função:

```
cumprimentar("Ada"); // → Olá, Ada! Seja bem-vindo!
```

Declaração

É criada automaticamente. Pode ser chamada antes de ser definida no código.

Expressão

Não é criada. A variável existe, mas o valor (a função) só estará disponível após a atribuição.

Arrow Functions

Introduzidas em 2015 (ES6), as **arrow functions** oferecem uma sintaxe mais concisa usando o operador `=>`. A seta representa "esta entrada produz este resultado".

```
const media = (n1, n2) => { return (n1 + n2) / 2 };
```

1

Parênteses opcionais

Com apenas **um parâmetro**, os parênteses podem ser omitidos.

```
const quadrado = x => x * x;
```

2

Retorno implícito

Se o corpo é uma **única expressão**, as chaves e o `return` são dispensáveis.

```
const quadrado = x => x * x;
```

3

Sem parâmetros

Use um par de parênteses vazio quando não houver parâmetros.

```
const latir = () =>  
  console.log("Roof!");
```

Argumentos vs. Parâmetros

Esses dois termos são frequentemente confundidos, mas têm significados distintos e precisos. Entender a diferença é fundamental para comunicar ideias sobre código com clareza.

Parâmetro

A **variável** declarada na definição da função, que receberá um valor quando ela for invocada. É como uma "caixinha com nome" dentro da função.

```
function somar(a, b) { ... }  
// a e b são PARÂMETROS
```

Argumento

O **valor concreto** passado para a função no momento da invocação. É o dado real que preenche aquela "caixinha".

```
somar(10, 20);  
// 10 e 20 são ARGUMENTOS
```

Você passa **argumentos** → eles são recebidos em **parâmetros**.

Argumentos opcionais

JavaScript é extremamente tolerante quanto ao número de argumentos passados para uma função. Isso pode ser uma vantagem — ou uma armadilha para os desatentos.



Argumentos extras

Se você passar **mais argumentos** do que a função possui parâmetros, os excedentes são simplesmente **ignorados**. Nenhum erro é disparado.



Argumentos faltando

Se você passar **menos argumentos** do que o esperado, os parâmetros não preenchidos recebem o valor `undefined` automaticamente.



Flexibilidade intencional

Esse comportamento permite criar funções que aceitam **diferentes números de argumentos**, como o próprio `console.log()`, que exibe quantos valores quisermos.



A ausência de verificação de aridade pode esconder bugs silenciosos. Use com consciência!

Valor padrão de parâmetros

Para evitar que parâmetros recebam `undefined` quando não fornecidos, o JavaScript permite definir **valores padrão** usando o operador `=` após o nome do parâmetro. Se o argumento for omitido na chamada, o valor padrão é usado automaticamente.

```
function saudacao(nome = "visitante") {  
  return console.log(`Olá, ${nome}!`);  
}
```

```
saudacao("Ada"); // → Olá, Ada!  
saudacao();      // → Olá, visitante!
```

Isso permite escrever funções mais **robustas e expressivas**, que funcionam corretamente mesmo quando alguns argumentos não são fornecidos pelo código que as chama.

 `console.log` é um exemplo clássico de função sem número fixo de argumentos: aceita qualquer quantidade de valores e os exibe em sequência.

Parâmetros restantes

Se os valores padrão permitem chamar funções com **menos argumentos**, os parâmetros restantes (*rest parameters*) permitem o oposto: funções chamadas com um número **arbitrariamente maior** de argumentos do que parâmetros declarados.

Sintaxe com ...

O parâmetro "rest" é precedido por **três pontos** (...) e deve ser sempre o **último parâmetro** da declaração da função.

Sempre um Array

Dentro da função, o valor de um parâmetro "rest" será **sempre um array** — nunca undefined. O array pode estar vazio, mas nunca é indefinido.

Sem valor padrão

Nunca é útil — nem válido — definir um **valor padrão** para um parâmetro "rest", pois ele sempre será um array (possivelmente vazio).

Parâmetros restantes: exemplo

A função abaixo recebe um ou mais números e retorna o maior deles, usando um parâmetro "rest" para coletar todos os argumentos adicionais em um array:

```
function max(primeiro = -Infinity, ...resto) {  
  let maiorValor = primeiro; // Começamos assumindo que o primeiro argumento é o maior da lista  
  
  // Percorre todos os argumentos restantes, procurando por maiores que o primeiro  
  for(let n of resto) {  
    if (n > maiorValor) {  
      maiorValor = n;  
    }  
  }  
  
  // Retorna o maior valor  
  return maiorValor;  
}  
max(1, 10, 100, 2, 3, 1000, 4, 5, 6) // => 1000
```

⚠ Não confunda o `...` de parâmetro rest com o **spread operator** `...`. Embora a sintaxe seja idêntica, eles operam em direções opostas: rest *coleta* valores em um array; spread *expande* um array em valores individuais.

Tipos de argumento

Diferentemente de linguagens como **Java** ou **TypeScript**, o JavaScript **não possui tipos declarados** nos parâmetros de funções. Nenhuma verificação automática de tipo é realizada nos valores passados.

Conversão automática

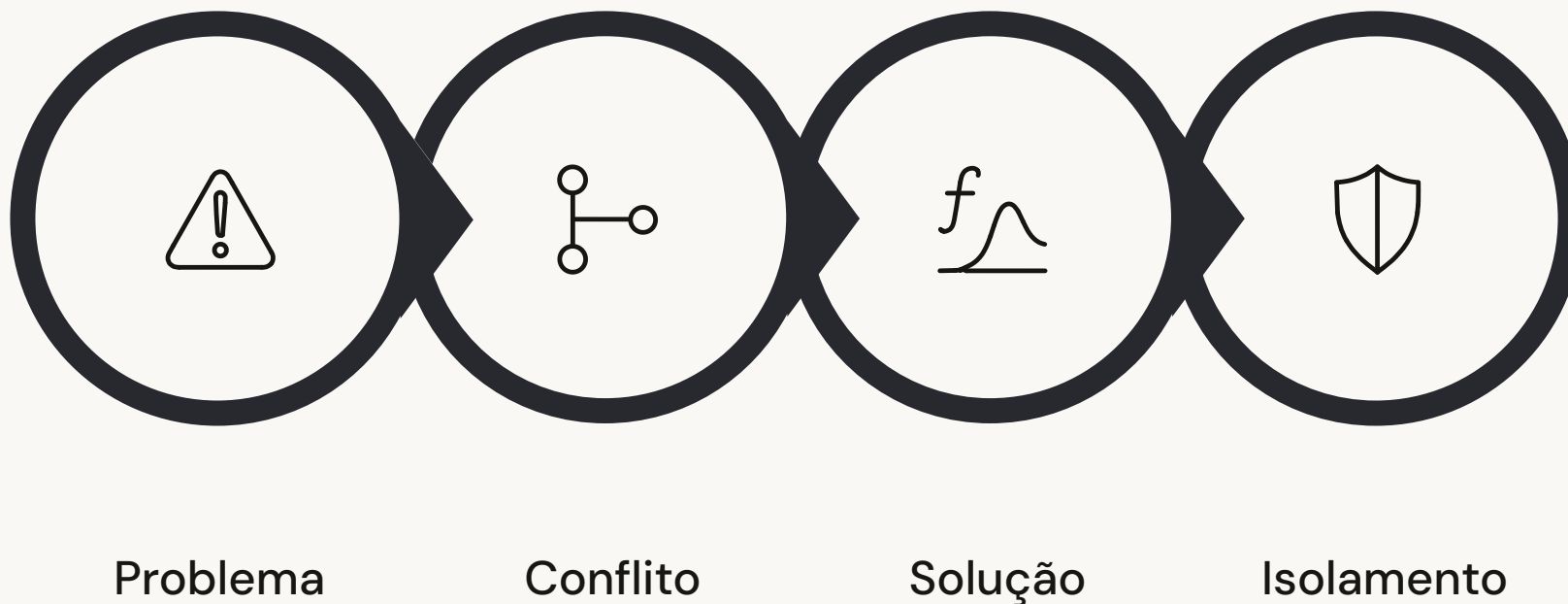
O JavaScript realiza **conversões de tipo flexíveis** conforme necessário. Se uma função espera uma string e recebe um número, ele será convertido automaticamente quando a função tentar usá-lo como string.

Boas práticas

- Escolha **nomes descritivos** para os parâmetros, tornando o código auto-documentado.
- Documente os tipos esperados em **comentários** de cada função.
- Considere usar **TypeScript** para projetos que exigem segurança de tipos.

Funções como Namespace

Variáveis declaradas dentro de uma função **não são visíveis** fora dela. Esse comportamento pode ser explorado intencionalmente: a função age como um **espaço de nomes** (*namespace*) temporário, isolando variáveis locais e evitando que poluam o escopo global.



Essa técnica é especialmente útil em trechos de código reutilizados em múltiplos programas ou páginas web, onde o risco de conflito de nomes é maior.

Funções como Namespace: Exemplo

Imagine que você tem um trecho de código reutilizável que cria variáveis para armazenar resultados intermediários. Para evitar conflitos com as variáveis de outros programas, basta envolvê-lo em uma função e invocá-la imediatamente:

```
function trechoEmNamespace() {  
  // Código no Namespace vai aqui  
  // Qualquer variável definida aqui será local a esta função  
  // ao invés de poluir o escopo global do programa  
}  
  
trechoEmNamespace(); // Não esqueça de invocar a função!
```

Essa ideia evoluiu para o padrão **IIFE** (*Immediately Invoked Function Expression*), muito usado antes do ES6 para criar escopos isolados em bibliotecas e módulos JavaScript.

Closures

Closures são um dos recursos mais **poderosos** do JavaScript. Uma closure é criada quando uma função interna "lembra" das variáveis do escopo da função externa, mesmo após a função externa ter encerrado sua execução.

```
function criarPartida(nomeDaPartida) {  
  let gols = 0;  
  return function marcarGol() {  
    gols++;  
    return `A partida ${nomeDaPartida} tem ${gols} gols marcados.`;  
  };  
}  
  
const classico = criarPartida('Corinthians x Palmeiras');  
classico(); // "A partida Corinthians x Palmeiras tem 1 gols marcados."  
classico(); // "A partida Corinthians x Palmeiras tem 2 gols marcados."
```

A função `marcarGol` **fecha sobre** as variáveis `nomeDaPartida` e `gols`, mantendo acesso a elas mesmo depois que `criarPartida` terminou de executar.



Considerações Finais

Este material cobre os aspectos essenciais das funções em JavaScript, mas é apenas o ponto de partida. O aprendizado contínuo — lendo, praticando e experimentando — é o que consolida o conhecimento.

01

Pratique o que aprendeu

Escreva funções com declaração, expressão e arrow functions. Experimente parâmetros opcionais, padrões e rest.

02

Aprofunde nos próximos tópicos

Explore **closures** avançadas, **higher-order functions** (map, filter, reduce) e **memoização**.

03

Consulte as referências

Leia o *Eloquent JavaScript* e o *JavaScript: The Definitive Guide* para maior profundidade técnica e exemplos ricos.

📌 Funções são o coração do JavaScript. Dominá-las é dominar a linguagem.

Referências

As fontes abaixo foram utilizadas na elaboração deste material e são altamente recomendadas para aprofundamento:

JavaScript: The Definitive Guide

FLANAGAN, David. **JavaScript: the definitive guide**. 7. ed. Sebastopol: O'Reilly Media, 2020. p. 200–237.

Eloquent JavaScript

HAVERBEKE, Marijn. **Eloquent JavaScript: a modern introduction to programming**. 4. ed. San Francisco: No Starch Press, 2024. cap. 3.

Closures — Wes Bos

BOS, Wes. **Closures**. Disponível em: wesbos.com. Acessado em: 02 ago. 2026.

MDN Web Docs — Funções

Mozilla Developer Network. **Funções**. Disponível em: developer.mozilla.org. Acessado em: 02 ago. 2026.